

Architecture and Workflow

This document is the implementation-level workflow reference for CVM++. It describes how source code moves through Driver, Lexer, Parser, Compiler, and VM, and maps each stage to its primary source files.

Workflow Overview

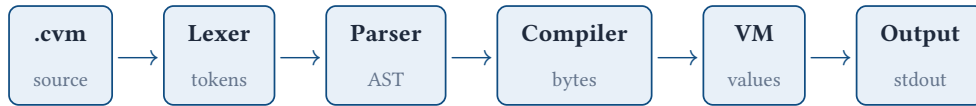


Figure 1: Pipeline: source → tokens → AST → bytecode → VM.

Stage	Input	Output	Primary file(s)
Driver	CLI flags and source path	source text or diagnostics	main.cpp
Lexer	source text	token stream	lexer.cpp
Parser	token stream	Program AST	parser.cpp, ast.hpp
Compiler	AST	BytecodeChunk	compiler.cpp, bytecode.cpp
VM	BytecodeChunk	stdout and status	vm.cpp

`compile_frontend()` in `compile.cpp` orchestrates `Lexer` → `Parser` → `Compiler`. If compile succeeds, `main.cpp` calls `execute()` in `vm.cpp` unless compile-only mode is enabled.



Figure 2: Compile time (front end) versus runtime (virtual machine).

CLI mode	Behavior
cvmpp file.cvm	Compile then run
-c	Compile only; VM does not run
-d	Print tokens, AST, and bytecode before running
-q	Hide informational banners

Driver Stage

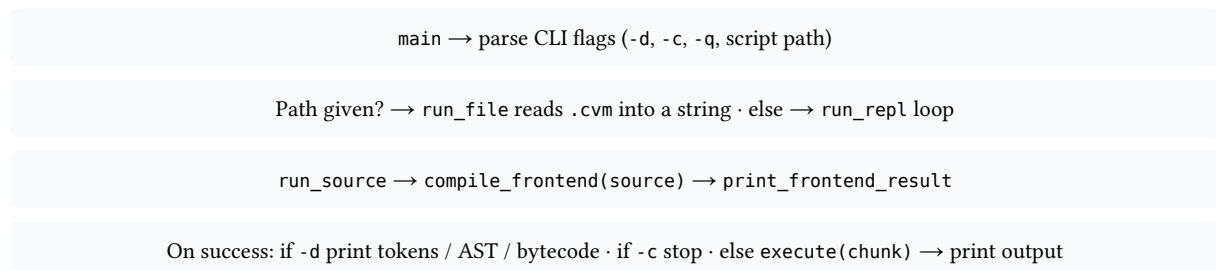


Figure 1: `main.cpp` control flow from `argv` to compile or run.

Responsibilities:

1. Parse flags and select file mode or REPL mode.
2. Read source into memory.
3. Run `compile_frontend()`.

4. Handle compile diagnostics and exit with code 1 on failure.
5. Run debug dumps (-d) and VM execution (unless -c).

When any compile stage fails, later stages are skipped:

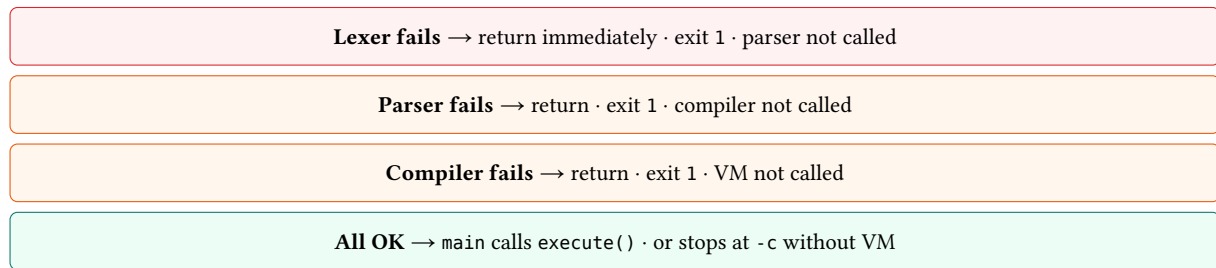


Figure 2: Where compile_frontend can stop — later stages never run after a failure.

Core Artifacts

Program AST

Parser output root is Program:

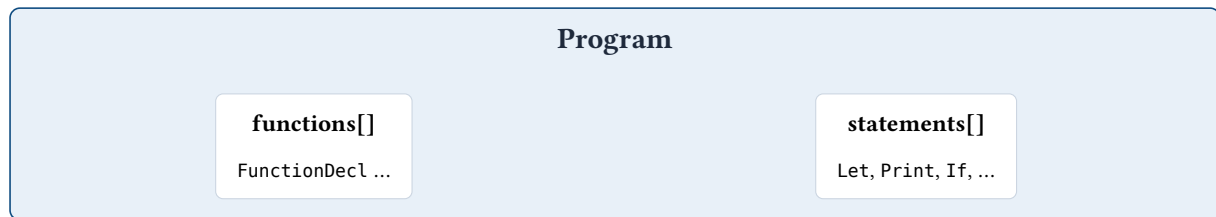


Figure 3: Parser output: one Program node with two lists — fn declarations vs everything else at top level.

Field	Meaning	Consumed by
functions[]	Top-level function declarations	Compiler function metadata and code emission
statements[]	Top-level executable statements	Compiler main-script emission

Bytecode chunk

Compiler output is BytecodeChunk:

Part	Type	Purpose
code[]	vector<uint8_t>	Raw instructions: opcode bytes + operands
names[]	vector<string>	Global names: index 0 → "x", index 1 → "flag"
functions[]	vector<FunctionMeta>	Each fn: name, start address, parameter count

Figure 1: BytecodeChunk — three parallel arrays the VM needs at runtime.



Metadata: names[], functions[] (name, address, arity).

Figure 3: Logical layout of BytecodeChunk::code after compilation (function-first placement).

Chunk component	Role
code[]	Opcode bytes plus encoded operands
names[]	Interned global-name pool referenced by slot
functions[]	Function metadata (entry offset, arity, identity)

Stage Reference

Lexer

Lexer::tokenize() scans source left-to-right and emits token records with:

1. token type,
2. lexeme,
3. source location (line, column).

Token class	Examples	Purpose
Keywords	let, fn, if, while, return, print	Language control and declarations
Identifiers	variable/function names	Symbol references
Literals	integers and booleans	Immediate values
Operators	+, -, *, <, <=, =	Expression and assignment semantics
Delimiters	() { } ; ,	Grammar structure
Terminator	Eof	End-of-input marker

Parser

Parser builds AST nodes using recursive descent with precedence-aware expression parsing. Parse diagnostics stop the pipeline before compilation.

Generic AST shape (not tied to one file):

```
Program
|-- functions[]
|  +-- FunctionDecl(name, params, body)
+-- statements[]
   +-- Let / Print / If / While / Return nodes
```

Compiler

Compiler lowers AST nodes into bytecode and metadata. It does not execute program logic.

Source form	Lowering pattern (typical)
Variable declaration	Evaluate expression → STORE_VAR
Print statement	Evaluate expression → PRINT
Conditional	Condition eval → JUMP_IF_FALSE around block
Loop	Header label → condition → exit jump → body → back jump
Function call	Evaluate args → CALL with function index and arity
Return	Evaluate expression → RETURN

loop start → compile condition → JumpIfFalse (exit)
body → Jump back to loop start → patch exit label

Figure 4: while lowering: condition at top of loop; exit jump patched after the body.

Virtual machine

VM executes a fetch-decode-execute loop over code[] until HALT or runtime fault.

VM state	Runtime role
ip_	Instruction pointer into code[]
stack_	Operand stack for expression evaluation
globals_	Global storage indexed by name slots
frames_	Call frames containing locals and return addresses



Figure 4: Function call: arguments move from operand stack to callee local slots.

Generic opcode flow (stack view):

IP	Opcode	Stack before	Stack after	Notes
n	LOAD_VAR a	[]	[value(a)]	Push variable value
n+1	LOAD_VAR b	[value(a)]	[value(a), value(b)]	Push second operand
n+2	ADD	[value(a), value(b)]	[value(a+b)]	Pop two, push result
n+3	PRINT	[value(a+b)]	[]	Pop and print
n+4	HALT	[]	[]	Stop execution

Function and Recursion Workflow

Function execution follows this runtime pattern:

1. Caller evaluates arguments and pushes them.
2. VM executes CALL and creates a new frame.
3. Callee reads parameters from frame-local slots.
4. Callee executes until RETURN.
5. VM restores caller frame and resumes at return address.

Frame field	Meaning
Local slot	Parameter/local value storage for the active frame
Return address	Instruction offset to continue after RETURN

Frame depth	Current call nesting level
-------------	----------------------------

Generic call-stack ladder:

Moment	Top frame	Caller frame	Event
Before CALL	none	current	Caller has evaluated arguments
After CALL	callee	caller	New frame created, locals initialized
During callee	callee	caller	Callee executes bytecode
At RETURN	callee	caller	Return value prepared
After RETURN	caller	previous caller	Callee frame popped, caller resumes

Diagnostics and Exit Behavior

Symptom	Likely stage	Primary file to inspect
Malformed tokenization	Lexer	lexer.cpp
Syntax error	Parser	parser.cpp
Incorrect jump layout / opcode sequence	Compiler	compiler.cpp
Wrong runtime value / stack behavior	VM	vm.cpp

Front-end early-stop rule:

```
result.lex = lexer.tokenize();
if (!result.lex.ok()) return result;
result.parse = parser.parse();
if (!result.parse.ok()) return result;
result.compile = compiler.compile();
return result;
```

Exit code	Meaning
0	Success (HALT reached)
1	Compile-time failure (Lexer, Parser, or Compiler)
2	Runtime VM failure

Opcode Reference

0x01–0x03 Literals / stack PushInt PushBool Pop
0x10–0x13 Variables LoadVar StoreVar LoadLocal StoreLocal
0x20–0x2A Math / compare Add...Div Eq Ne Lt Gt Neg Le Ge
0x30–0x31 I/O Input Print
0x40–0x43 Control / calls Jump JumpIfFalse Call Return
0xFF Stop Halt

Figure 5: Opcode groups in `opcode.hpp` (byte tag + fixed operands).